

Objetivo, alcance y preparación

Control

Versión 2.0 | Corte documental: 29/09/2026. Documento técnico para revisión académica. Implementado significa observado en código; no equivale a validación productiva. Las iteraciones son planes de trabajo, no actas ejecutadas.

Objetivo

Asignar y reasignar sin duplicados, respetando plazos y confirmación.

Situación observada

Algoritmo y ciclo implementados; integración real, concurrencia y calibración local pendientes.

Dependencias

I01-I03; fixture y reglas versionados.

Historias de usuario

ID	Historia
HU-07	Como cliente quiero recibir una propuesta de un técnico elegible.
HU-08	Como técnico quiero responder antes del vencimiento sin doble asignación.

Trazabilidad y antecedentes

O2-O4; RF-06 a RF-19, RF-23. A03, A05, A06, A15. Las referencias [Axx] remiten a las fichas del documento 01 y al catálogo trazabilidad/v2/articulos.csv. Páginas citadas: numeración física del PDF, salvo indicación expresa. Los originales se conservan en fuentes/articulos/Axx.pdf; SHA-256 en fuentes/articulos/manifest.json.

Incluye

Ofertas pending/accepted/declined/expired/cancelled; no reiniciar pending_until al reasignar. Agreed_fee fija condición. Dirección solo asignado después de confirmación.

No incluye / riesgo principal

El score usa zonas, no GPS. La agenda del ejemplo se adapta a disponibilidad inmediata; reservas horarias quedan fuera del núcleo.

Entrada a desarrollo

Product Owner aprueba alcance y criterio; equipo dispone de entorno QA, datos sintéticos y dependencias. Definir capacidad del sprint antes de comprometer fechas. Si esta iteración es propuesta, la documentación no significa aprobación de implementación.

Iteración 4: Matching, disponibilidad y ciclo de solicitudes

Implementación por capas

Frontend

Mostrar estado y plazo real en RequestCard; no convertir contador en autoridad. Deshabilitar envío mientras procesa y mostrar 409 si otro actor ganó. Explicar ausencia de cobertura.

Backend Laravel

Marketplace::dispatch, tick y action conservan ventana global. Procesador cada segundo mediante schedule:work. Mantener bloqueo de solicitud y técnico. Evaluar idempotencia de create antes de reintento automático; hoy no se observa clave idempotente.

Python

POST /match ordena elegibles y devuelve rulesVersion. Laravel contrasta IDs y puntajes con fallback local; timeout total 2 s, conexión 1 s. Probar caída, JSON inválido y versión divergente.

Base de datos y contrato

Ofertas pending/accepted/declined/expired/cancelled; no reiniciar pending_until al reasignar. Agreed_fee fija condición. Dirección solo asignado después de confirmación.

Secuencia para desarrollar

1. Reproducir situación actual y anotar commit. 2. Escribir caso negativo o límite que falle. 3. Implementar cambio mínimo en capa responsable. 4. Verificar respuesta y persistencia. 5. Refactorizar sin cambiar contrato. 6. Ejecutar regresión y revisar con otro integrante. 7. Actualizar evidencia y desplegar QA.

Errores y seguridad

La API vuelve a validar permisos y datos aunque la UI bloquee una acción. No usar localStorage como control real. Mantener mensajes comprensibles, datos de prueba sintéticos y secretos fuera de capturas/logs. No registrar un cambio como completo si solo existe en el frontend.

Pruebas, entrega y finalización

Casos de aceptación

ID	Procedimiento	Resultado esperado
I04-01	Cuatro técnicos elegibles al crear	Como máximo tres ofertas pendientes
I04-02	Aceptar en 10 minutos exactos	Oferta rechazada como vencida
I04-03	Avanzar reloj hasta 24 horas	unassigned; ninguna oferta pendiente
I04-04	Dos aceptaciones simultáneas en PostgreSQL	Una sola asignación; perdedor recibe conflicto
I04-05	Python caído o ranking alterado	Fallback PHP coincide con fixture

Ejecución TDD

RED: ejecutar el caso que demuestra la falta o fallo, conservar resultado. GREEN: implementar y repetir. REFACTOR: limpiar código y ejecutar regresión pertinente. Para funciones ya implementadas, registrar prueba de caracterización y no inventar una fase RED anterior.

Despliegue de la iteración

Desplegar API y Python juntos en QA; ejecutar worker de plazos supervisado y reiniciarlo durante una solicitud de prueba. Verificar recuperación sin reiniciar ventanas.

Criterios de salida

Historias aceptadas, permisos negativos aprobados, contrato y diccionario actualizados, pruebas pertinentes sin fallos y QA verificable. Guardar build/commit, entorno, caso y observado. Riesgos pendientes con responsable por asignar; ninguna firma ni aceptación simulada.

Carpeta de evidencia

evidencias/v2/I04/ contiene README de estado. En la ejecución se añadirán logs, casos, incidentes, retest, acta de revisión y registro de despliegue. Actualmente el estado es PLANIFICADO / NO EJECUTADO como iteración. No confundir pruebas previas con cierre de sprint.